

Event Based Camera (EBC Lab)

Lab 2

Course: Hands-on Perception

Date: April 4, 2025

Pravin Oli

u1999699, pravin.oli.08@gmail.com

Gebreslassie Gebrecherkos Gebrezgabhier

u1999542, chereg2016@gmail.com

Abstract

This lab focuses on implementing motion compensation on event-based data using a pyramidal patch-based optimization technique. Raw event streams from the "indoor flying1" dataset are processed to align motion trajectories and improve the temporal consistency of event images. Performance is quantitatively evaluated using the Average Endpoint Error (AEE). We analyze the impact of different sequence durations and patch scale initializations on convergence and accuracy. Results demonstrate that motion compensation significantly enhances visual alignment and that selecting an appropriate time window and scale range is crucial for optimal performance.

Contents

1	Introduction	2
2	Objectives	2
3	Implementation	2
3.1	Environment Setup	2
3.2	Sequence Clipping	2
3.3	Event and Flow Cropping	3
3.4	Motion Compensation (Base Case)	3
3.5	Patch Initialization: Coarse vs Fine Scales	3
3.6	Metric Computation	3
3.7	Visualization	3
4	Results and Discussion	3
5	Conclusion	9

1 Introduction

In this lab, we explored motion compensation techniques for event-based data using sequences from the indoor flying1 dataset. Event-based cameras asynchronously capture changes in brightness, which can appear noisy or misaligned during motion. To address this, we applied a pyramidal patch-based motion compensation algorithm that estimates pixel-wise flow and aligns the events over time. This improves the clarity of event representations and enables better analysis of motion, particularly for high-speed scenes.

2 Objectives

- Apply a pyramidal patch-based motion compensation method to improve the alignment of asynchronous events.
- Assess the accuracy of the compensated optical flow using the Average Endpoint Error (AEE) metric.
- Compare the impact of different temporal slices (short, medium, long) on the algorithm's convergence and performance.
- Analyze how different pyramid scale settings (e.g., (1, 5) and (4, 5)) affect the quality of motion compensation results.

3 Implementation

The motion compensation pipeline was implemented using the provided motion tutorial.py script and followed the steps outlined in the lab instruction:

3.1 Environment Setup

A dedicated Conda environment was created and all dependencies were installed using the provided requirements.txt file. The dataset and ground truth flow files were loaded using h5py, and the ewiz library was used to handle event data processing, flow synchronization, and loss computation.

3.2 Sequence Clipping

We selected sequences from the indoor flying1 dataset and adjusted the clip variable accordingly. Three main cases were tested:

- **Base Sequence:** From 10.0s to 10.2s
- **Short Sequence:** From 10.0s to 10.015s
- **Long Sequence:** From 20.0s to 23.2s

The ground truth flow was clipped to match the sequence, and rescaled by the duration of the sequence for normalization.

3.3 Event and Flow Cropping

To reduce computational complexity and ensure input compatibility, event data and ground truth flows were center-cropped to a fixed size of (256, 256) :

3.4 Motion Compensation (Base Case)

The pyramidal patch-based optimizer was initialized using a BFGS method with random initialization and scale range (1, 5). This optimizer performed iterative refinement of optical flow across scales to minimize the motion compensation loss. We recorded both the loss value and the Average Endpoint Error (AEE) after the run.

3.5 Patch Initialization: Coarse vs Fine Scales

To study the effect of scale resolution, we repeated the same experiment for the base sequence (10.0s - 10.2s) using a tighter `scale_range=(4, 5)`.

3.6 Metric Computation

We implemented the AEE metric using the provided function stub. The AEE was computed by masking out invalid flow regions and measuring the average L2 norm between predicted and ground truth flows.

3.7 Visualization

The following visualizations were captured for comparison:

- Events Image (Before Warping)
- Events Image (After Warping)
- Ground Truth Flow
- Compensated Flow

4 Results and Discussion

Part 3: Motion Compensation

In this section, we used the ‘motion_tutorial.py’ script to apply patch-based motion compensation using the eWiz library. The pipeline was tested on sequences clipped from the “indoor_flying1” dataset. All results are evaluated with both loss and AEE metrics.

Sequence 1: [10.0s – 10.2s]

Optimizer Settings: `scale_range=(1,5)`

Iterations: 80

Function evaluations: 111

Gradient evaluations: 111

- **Final Loss:** 3.65197
- **AEE:** 39.5687

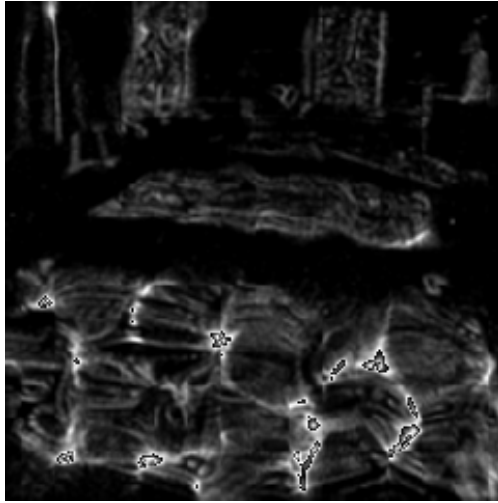


Figure 1: IWE for sequence 10.0s – 10.2s, scale range=(1,5)

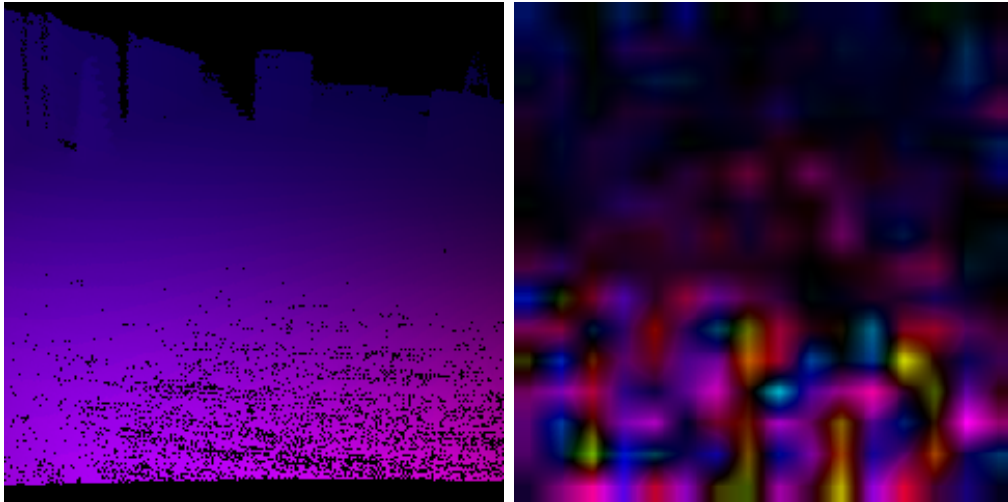


Figure 2: Left: Ground Truth Flow, Right: Compensation Flow (for sequence 10.0s – 10.2s, scale range=(1,5))

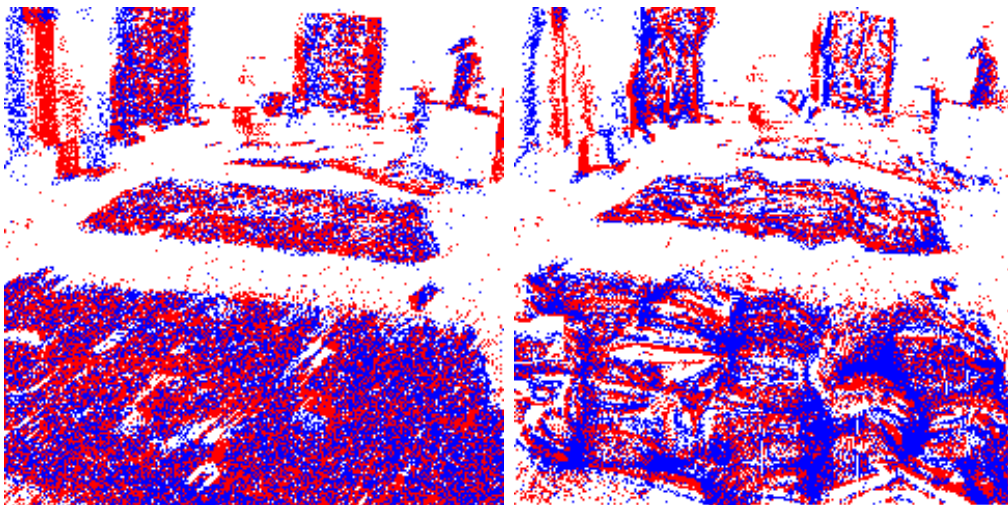


Figure 3: Left: Events image before warping, Right: Events image after motion compensation (for sequence 10.0s – 10.2s, scale range=(1,5))

Answers to Lab Questions

1. Compare the Events Images (before and after warping)

The events image **before warping** appears blurry due to accumulated motion across time. The **after warping** image is much sharper and better aligned, as the events are temporally compensated using the predicted flow field. The improvement is visual evidence of motion alignment.

2. How does the IWE change during optimization?

The Image of Warped Events (IWE) becomes increasingly sharp through the optimization steps. Initially, the event clusters are scattered. As the optimizer updates the flow, the events align better, increasing image contrast and structure. This is reflected in the declining loss values.

Case Study 1: Short Sequence

Sequence: 10.0 to 10.015 seconds

Optimizer Settings: scale_range=(1,5)

Iterations: 80

Function evaluations: 81

Gradient evaluations: 81

Results:

- **Final Loss:** 8.0004
- **AEE:** 60.7247



Figure 4: IWE for sequence 10.0s – 10.015s, scale range=(1,5)

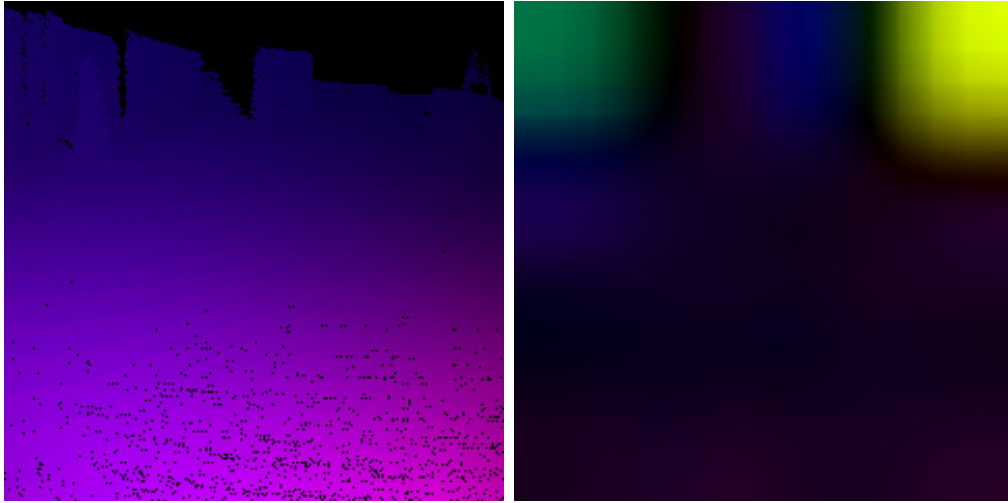


Figure 5: Left: Ground Truth Flow, Right: Compensation Flow (for sequence 10.0s – 10.015s, scale range=(1,5))

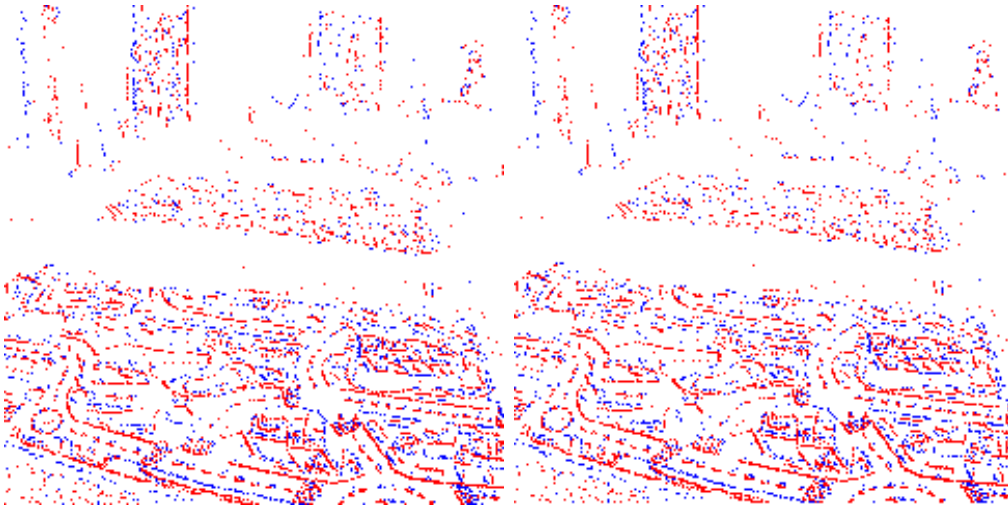


Figure 6: Left: Events image before warping, Right: Events image after motion compensation (for sequence 10.0s – 10.015s, scale range=(1,5))

Observation: The algorithm did not converge well. The short time interval led to insufficient event information, making flow estimation unreliable and noisy.

Case Study 2: Long Sequence

Sequence: 20.0 to 23.2 seconds

Optimizer Settings: scale_range=(1,5)

Iterations: 80

Function evaluations: 82

Gradient evaluations: 82

Results:

- **Final Loss:** 1.3856
- **AEE:** 12.7466

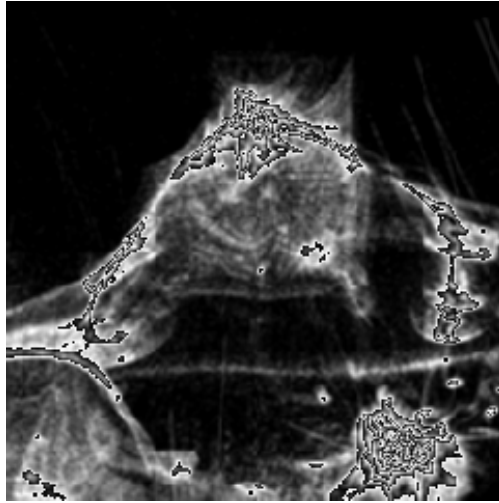


Figure 7: IWE for sequence 20.0s – 23.2s, scale range=(1,5)

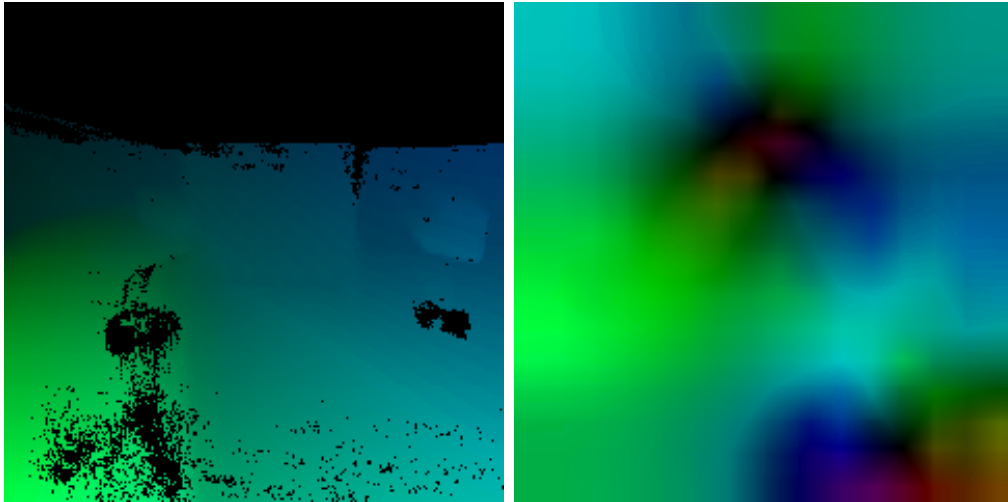


Figure 8: Left: Ground Truth Flow, Right: Compensation Flow (for sequence 20.0s – 23.2s, scale range=(1,5))

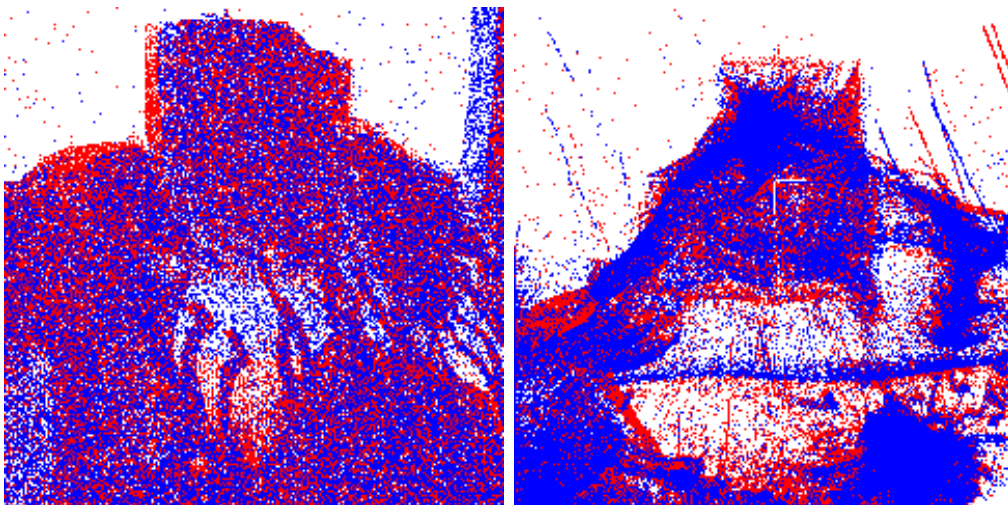


Figure 9: Left: Events image before warping, Right: Events image after motion compensation (for sequence 20.0s – 23.2s, scale range=(1,5))

Observation: The algorithm performed moderately well. The long sequence provided more motion data but may have introduced drift or over-smoothing.

Patch Initialization Test

We changed the pyramidal optimizer scale range to (4,5) for a finer-grained initialization.

Sequence: 10.0 to 10.2 seconds

Optimizer Settings: scale_range=(4,5)

Iterations: 80

Function evaluations: 101

Gradient evaluations: 101

Results:

- **Final Loss:** 3.4364
- **AEE:** 37.3479

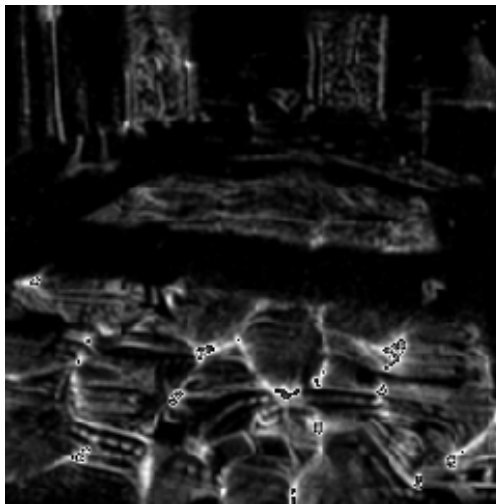


Figure 10: IWE for sequence 10.0s – 10.2s, scale range=(4,5)

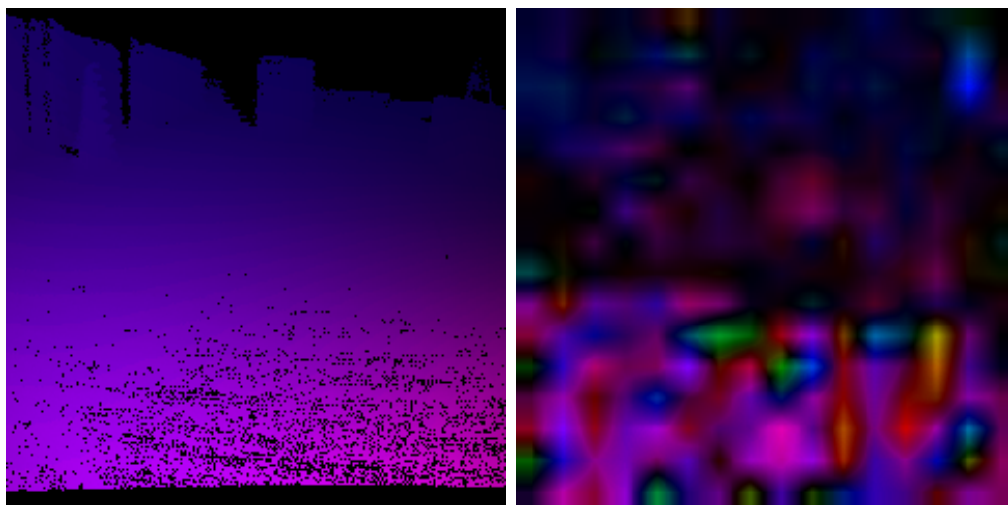


Figure 11: Left: Ground Truth Flow, Right: Compensation Flow (for sequence 10.0s – 10.2s, scale range=(4,5))

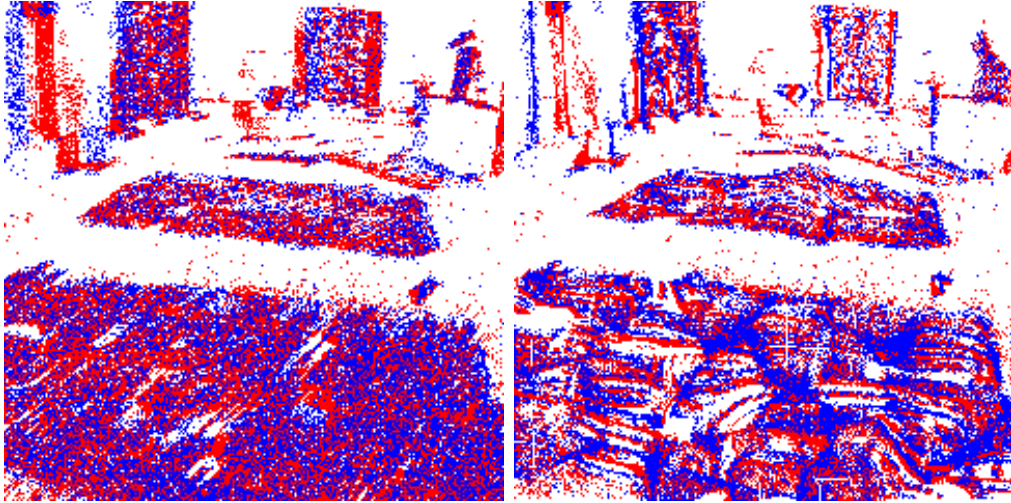


Figure 12: Left: Events image before warping, Right: Events image after motion compensation (for sequence 10.0s – 10.2s, scale range=(4,5))

Observation: The finer initialization resulted in faster convergence, but overall AEE increased due to reduced global context.

5 Conclusion

In this lab, we explored motion compensation in event based vision by applying a pyramidal patch-based optimizer on raw event data. Our results demonstrated that motion compensation significantly improves the visual alignment of events and allows for better motion estimation. The comparison of event images before and after warping showed enhanced clarity and sharper motion boundaries.

From the case studies: we concluded that, **Short sequences** lacked sufficient motion information, resulting in poor flow estimation and high AEE values. **Long sequences**, while rich in motion, introduced optimization challenges such as overfitting or flow drift, leading to suboptimal convergence. Using a **fine-scale patch initialization** improved convergence speed and local detail but could compromise global accuracy.